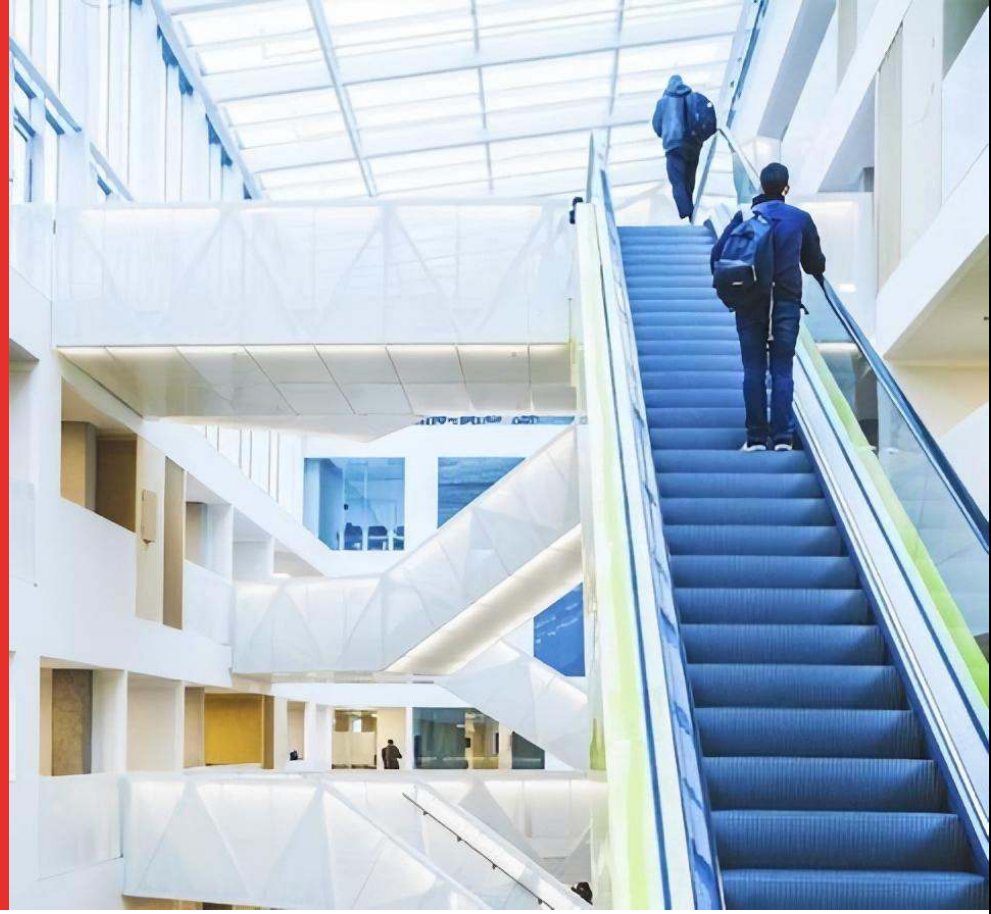
OVERZICHT LES

- Complexiteit
- Wat is een algoritme?
- Vergelijking C++/Python



An error occurred on this slide. Check the terminal for more information.

TWEE ALGORITMES, WELKE IS BETER?

Ik zeg:

- Max1 is beter, want je ziet toch dat het sneller is?

Maar wat bedoelen we precies met "beter"?

- Runtime: Hoe snel het algoritme werkt bij groeiende input.
- Programmeertaal en Compiler: De efficiëntie kan afhangen van de taal en optimalisaties.
- Implementatie: Kleine verschillen in implementatie (bijv. gebruik van iterators vs. indexering) kunnen effect hebben.

TWEE ALGORITMES, WELKE IS BETER?

Kunnen we dat kwantificeren?

- We kunnen meten hoeveel CPU-instructies er uitgevoerd worden.
- Maar belangrijker is de asymptotische groei, hoeveel X langzamer wordt mijn functie als de input veranderd.
- Dit vertelt ons hoe het algoritme schaalt als de inputgrootte toeneemt.

HET BETERE ALGORITME

Wat boeit ons echt? Hoe snel wordt de runtime groter naarmate de input toeneemt?

- Constante factoren worden in de Big-O notatie genegeerd.
- We richten ons op de worst-case performance.
- Dit is essentieel voor grote inputs, waar de groeiende factor bepalend is.

HET BETERE ALGORITME

Om de efficiëntie van een algoritme te bepalen:

- Analyseer de worst-case scenario's.
- Vergelijk de orde van groei:
  - O(1) is constant.
  - O(N) groeit lineair.
  - $O(N^2)$ groeit kwadratisch.

Deze abstractie helpt ons te bepalen welk algoritme beter schaalt, ongeacht de precieze constante factoren.

TIJD COMPLEXITEIT

Big-O notatie geeft een abstracte maat voor de groei van de runtime als functie van de inputgrootte:

- O(1): Constante tijd
- O(N): Lineaire tijd
- $O(N^2)$: Kwadratische tijd
- O(log N): Logaritmische tijd
- O(N log N): Linearitmische tijd
- O(2^N): Exponentiële tijd
- O(N!): Faculteit tijd

Kleine optimalisaties of constante overhead tellen niet mee. We kijken naar de orde van de groei.

VOORBEELD O(N) - LINEAIR

C++:

```cpp
#include <vector>
#include <iostream>

int sumElements(const std::vector<int>& vec) {
    int sum = 0;
    for (int x : vec) {
        sum += x;
    }
    return sum;
}

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    std::cout << "Sum: " << sumElements(v) << "\n"
    return 0;
}
```

Python:

```python
def sum_elements(lst):
    total = 0
    for x in lst:
        total += x
    return total

lst = [1, 2, 3, 4, 5]
print("Sum:", sum_elements(lst))
```

VOORBEELD O(N²) - BUBBLE SORT

C++:

```cpp
#include <vector>
#include <iostream>
#include <algorithm>

void bubbleSort(std::vector<int>& vec) {
    int n = vec.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (vec[j] > vec[j+1]) {
                std::swap(vec[j], vec[j+1]);
            }
        }
    }
}

int main() {
    std::vector<int> v = {5, 2, 9, 1, 5, 6};
    bubbleSort(v);
    for (int x : v) {
        std::cout << x << " ";
    }
```

Python:

```python
def bubble_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lst[j] > lst[j+1]:
                lst[j], lst[j+1] = lst[j+1], lst[j]
    return lst

lst = [5, 2, 9, 1, 5, 6]
print("Sorted:", bubble_sort(lst))
```

VOORBEELD O(LOG N) - BINARY SEARCH

C++:

```cpp
#include <vector>
#include <iostream>

int binarySearch(const std::vector<int>& vec, int t
    int low = 0, high = vec.size() - 1;
    while (low ≤ high) {
        int mid = low + (high - low) / 2;
        if (vec[mid] == target) {
            return mid;
        } else if (vec[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main() {
```

Python:

```python
def binary_search(lst, target):
    low, high = 0, len(lst)-1
    while low ≤ high:
        mid = (low + high) // 2
        if lst[mid] == target:
            return mid
        elif lst[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

lst = [1, 3, 5, 7, 9, 11]
target = 7
print("Index:", binary_search(lst, target))
```


STL HUISWERK TERUGBLIK

- Lees alle karakters van deze file in, sla op in een geschikte container
- Tel en print hoeveel karakters de file bevat

```cpp
std::ifstream ifs{"bee.txt"};
std::string text{std::istreambuf_iterator<char>{
            std::istreambuf_iterator<char>{}};

std::println("Aantal karakters: {}", text.size());
```

```python
with open('bee.txt') as f:
    text = f.read()

print('Aantal karakters:', len(text))
```

STL HUISWERK TERUGBLIK

- Print hoeveel regels de file bevat
- Print hoeveel alfabetische karacters de file bevat

```cpp
auto line_count = ranges::count(text, '\n');
std::println("Aantal regels: {}", line_count);

auto alpha_count = ranges::count_if(text, [](unsig
   return std::isalpha(c);
});
std::println("Aantal alfabetische karakters: {}", a
```

```python
print('Aantal regels:', text.count('\n'))

print('Aantal alphabetische karakters:',
    sum(c.isalpha() for c in text))
```

STL HUISWERK TERUGBLIK

- Print of de file alleen maar letters en leestekens bevat
- Zet alle letters in de container om in kleine letters

```cpp
bool only_valid = ranges::all_of(text, [](unsigned c
    return std::isprint(c) || std::isspace(c);
});
std::println("Bevat de file alleen letters, spaties
          only_valid ? "Ja" : "Nee");

for (char& c : text) c = std::tolower(c);
```

```python
print('Bevat de file alleen letters, spaties en lees
    all(c.isprintable() or c.isspace() for c in text

text = text.lower()
```

STL HUISWERK TERUGBLIK

- Tel in een geschikte container voor iedere letter (a…z) hoe vaak deze voorkomt in de tekst Druk deze container af: gesorteerd op lettervolgorde gesorteerd op hoe vaak een letter voorkomt

```cpp
std::vector letter_freq(std::from_range, views::
  | views::transform([&](char l) { return std::pa

for (char c : text
        | views::filter([](unsigned char c){ retur
  letter_freq[c - 'a'].second++;

// gesorteerd op lettervolgorde
std::println("\nLetterfrequentie (alfabetische vol
for (const auto &p : letter_freq)
  std::println("{}: {}", p.first, p.second);

// gesorteerd op hoe vaak een letter voorkomt
ranges::sort(letter_freq, {}, &std::pair<char, siz

std::println("\nLetterfrequentie (oplopend):");
for (const auto& p : letter_freq)
  std::println("{}: {}", p.first, p.second);
```

```python
frequencies = {c: text.count(c) for c in ascii_lowe
print('\nLetterfrequentie (alphabetische volgorde)
for c in ascii_lowercase:
  print(c, frequencies[c], sep=': ')

print('\nLetterfrequentie (oplopend):')
for c, freq in sorted(frequencies.items(), key=lan
  print(c, freq, sep=': ')
```

STL HUISWERK TERUGBLIK

- Bepaal welke woorden er in de tekst voorkomen
- Druk de 10 meest voorkomende woorden af

```cpp
const auto is_word_char = [](char c) { return std:

auto words_view = text
   | views::chunk_by([&](auto a, auto b) {return i
   | views::filter([](auto r) { return std::isalpha
   | views::transform([](auto r) { return std::st

std::unordered_map<std::string_view, size_t> wor
for (const auto word : words_view) ++word_freq[w

const size_t top_n{10};

std::vector<std::pair<std::string_view, size_t>> t
ranges::partial_sort(top_words, top_words.begin(

std::println("\nTop {} meest voorkomende woorder
for (const auto [word, freq] : top_words | views::
   std::println("{:>10} : {:5}", word, freq);
```

```python
top_n = 10
print(f'\nTop {top_n} meest voorkomende woorden
words = text.split()
word_frequencies = {word: words.count(word) for
for word, freq in sorted(word_frequencies.items(
   print(f'{word:>10} : {freq>5}')
```

HUISWERK

ALDS opdracht notebook

\0